

Proyecto final: Clasificación Automática de Géneros Musicales

ABSTRACT

Este proyecto desarrolla un sistema inteligente para el juego Othello (Reversi) mediante la implementación de agentes basados en Minimax, Alpha-Beta Pruning y Monte Carlo Tree Search (MCTS), apoyados por una función heurística adaptativa que considera movilidad, control de esquinas, bordes y ventaja material. Se construyó además un motor completo del juego y una interfaz gráfica en Pygame para la interacción y visualización de partidas. La evaluación experimental, realizada mediante torneos automáticos, mostró que Alpha-Beta obtuvo el mejor desempeño con 17 victorias en 20 partidas frente a MCTS, además de requerir menor costo computacional. Asimismo, se comprobó que la poda Alpha-Beta reduce significativamente la cantidad de nodos explorados respecto a Minimax sin afectar la calidad de las decisiones, demostrando la efectividad de las técnicas de búsqueda adversarial y evaluación heurística en juegos de estrategia de información perfecta.

INDEX TERMS

Artificial Intelligence, Othello, Reversi, Minimax, Alpha-Beta Pruning, Monte Carlo Tree Search, Heuristic Evaluation, Adversarial Search, Game Playing.

I. ANTECEDENTES

La clasificación automática de géneros musicales es un problema de reconocimiento de patrones en señales de audio que ha ganado relevancia con el crecimiento de plataformas de *streaming* musical. Servicios como Spotify o Apple Music requieren sistemas capaces de organizar y recomendar contenido de manera eficiente, lo que hace de la categorización automática una herramienta fundamental para la gestión de grandes catálogos.

Desde una perspectiva matemática, este problema se formula como una tarea de clasificación multiclase, donde cada instancia es un vector de características acústicas $\mathbf{x} \in \mathbb{R}^d$ y la tarea consiste en aprender $f : \mathbb{R}^d \rightarrow \mathcal{Y}$, con $|\mathcal{Y}| = 10$ géneros.

Las características acústicas más empleadas provienen del análisis espectral y temporal de la señal: coeficientes cepstrales en frecuencias de mel (MFCC), centroide espectral, tasa de cruce por cero, *rolloff* espectral y tempo. El conjunto de datos GTZAN [gtzan2002], introducido por Tzanetakis y Cook en 2002, se ha convertido en el *benchmark* estándar para esta tarea, con 1,000 fragmentos de 30 segundos distribuidos en 10 géneros.

II. OBJETIVOS

II-A. OBJETIVO GENERAL

Desarrollar y comparar modelos de aprendizaje supervisado para la clasificación automática de géneros musicales, evaluando el impacto del origen de las características acústicas sobre el rendimiento de los clasificadores.

II-B. OBJETIVOS ESPECÍFICOS

- Implementar y comparar cinco modelos supervisados sobre el dataset GTZAN.
- Realizar un análisis exploratorio de las características acústicas.
- Implementar un pipeline de extracción de features con `librosa`.
- Comparar el rendimiento entre features del dataset y features extraídas manualmente.
- Desarrollar una interfaz Streamlit para clasificar canciones en tiempo real.

III. HERRAMIENTAS Y METODOLOGÍA

III-A. HERRAMIENTAS

El estudio fue implementado en Python 3 con las siguientes librerías: **pandas** y **numpy** para manipulación de datos; **scikit-learn** para modelos y métricas; **XGBoost** para *gradient boosting*; **librosa** para análisis de audio; **matplotlib** y **seaborn** para visualización; **Streamlit** para la interfaz de usuario; y **joblib** para serialización de modelos.

III-B. CONJUNTO DE DATOS

Se utilizó la versión tabulada `features_30_sec.csv` del dataset GTZAN, con 1,000 instancias y 57 características acústicas pre-calculadas para segmentos de 30 segundos. Las features incluyen estadísticas descriptivas (media y varianza) de 13 coeficientes MFCC, centroide espectral, ancho de banda espectral, *rolloff*, tasa de cruce por cero, energía RMS y tempo. Las 10 clases presentan 100 instancias cada una, constituyendo un problema balanceado.

III-C. PREPROCESAMIENTO

El pipeline incluyó: eliminación de columnas `filename` y `length`; codificación de la etiqueta con `LabelEncoder`; división 80/20 estratificada (`random_state=42`); y normalización con `StandardScaler` para los modelos sensibles a escala (Logistic Regression y Naive Bayes).

III-D. MODELOS DE CLASIFICACIÓN

III-D1. Random Forest

Random Forest construye B árboles sobre muestras bootstrap y combina sus predicciones por votación mayoritaria:

$$\hat{y} = \text{modo}\{T_b(\mathbf{x})\}_{b=1}^B$$

La aleatoriedad en la selección de features en cada nodo reduce la correlación entre árboles, mejorando la generalización. El modelo fue optimizado con `RandomizedSearchCV` (50 iteraciones, 3-fold estratificado) maximizando *balanced accuracy*.

III-D2. Decision Tree

Particiona el espacio de features mediante reglas binarias usando el criterio de impureza de Gini:

$$\text{Gini}(t) = 1 - \sum_{k=1}^K p_k^2$$

donde p_k es la proporción de la clase k en el nodo t . Se configuró con profundidad máxima de 10 para controlar el sobreajuste.

III-D3. XGBoost

Construye modelos de forma aditiva minimizando una función de pérdida regularizada en cada iteración m :

$$\mathcal{L}^{(m)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(m-1)} + h_m(\mathbf{x}_i)) + \Omega(h_m)$$

donde $\Omega(h_m)$ penaliza la complejidad del árbol. Se usó `multi:softmax` con `mlogloss` como métrica de evaluación, 200 estimadores y `learning_rate=0.05`.

III-D4. Naive Bayes Gaussiano

Asume independencia condicional entre features dado el género y modela cada característica con distribución normal:

$$P(\mathbf{x} | y=k) = \prod_{j=1}^d \frac{1}{\sqrt{2\pi\sigma_{kj}^2}} \exp\left(-\frac{(x_j - \mu_{kj})^2}{2\sigma_{kj}^2}\right)$$

La predicción se obtiene por la regla MAP sobre las probabilidades a *posteriori*.

III-D5. Regresión Logística

Extiende el modelo binario a K clases mediante la función *softmax*:

$$P(y=k | \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

Los parámetros se estimaron minimizando la entropía cruzada con el optimizador L-BFGS (`max_iter=5000`).

III-E. EXTRACCIÓN MANUAL DE CARACTERÍSTICAS

En la segunda fase se implementó un pipeline con `librosa` extrayendo: 13 MFCCs (media y varianza), *chroma STFT*, centroide espectral, ancho de banda espectral, *rolloff*, tasa de cruce por cero, energía RMS y tempo. Este nuevo dataset se usó para reentrenar Random Forest, XGBoost y Logistic Regression, permitiendo comparar el efecto de la fuente de features sobre el rendimiento.

III-F. EVALUACIÓN

Los modelos se evaluaron con: **Accuracy**, **Balanced Accuracy**, **F1-Score macro**, **ROC-AUC** (OvR ponderado) y **Matriz de confusión**. Para Random Forest se realizó adicionalmente validación cruzada de 3 *folds* sobre el conjunto de entrenamiento.

III-G. REPRESENTACIÓN ESPECTRAL DE LA SEÑAL

Antes de la extracción de características, cada señal de audio fue transformada a una representación tiempo-frecuencia mediante un espectrograma. Esta representación permite visualizar cómo se distribuye la energía de las diferentes frecuencias a lo largo del tiempo.

El espectrograma se obtiene mediante la Transformada de Fourier de Tiempo Corto (STFT), la cual divide la señal en ventanas temporales y calcula el contenido frecuencial de cada segmento. Matemáticamente, la STFT puede expresarse como:

$$X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n], w[n-m], e^{-j\omega n} \quad (1)$$

donde $x[n]$ representa la señal de audio original y $w[n]$ corresponde a una función ventana aplicada localmente sobre la señal.

A partir de esta representación se calcularon las características acústicas utilizadas por los modelos de clasificación, incluyendo MFCC, Spectral Centroid, Spectral Bandwidth, Spectral Rolloff y Chroma STFT. Estas características resumen propiedades relacionadas con el timbre, la energía, la armonía y el contenido espectral de las canciones.

La Figura 1 muestra un ejemplo de waveform y espectrograma utilizados durante el proceso de extracción de características.

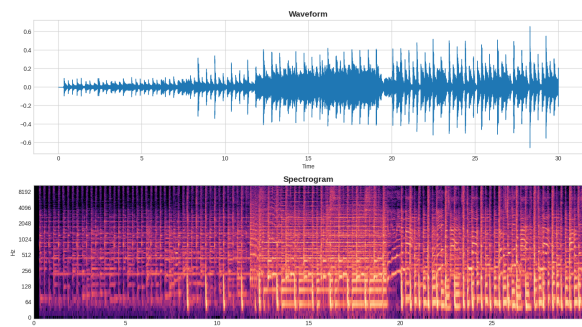


Figura 1: Ejemplo de waveform y espectrograma de una canción procesada mediante librosa.

IV. RESULTADOS

IV-A. ANÁLISIS EXPLORATORIO

El dataset presenta 1,000 instancias sin valores nulos ni duplicados. Las distribuciones de las features muestran variabilidad considerable entre géneros, indicando que los descriptores acústicos son informativos para la tarea.

Distribución de Features

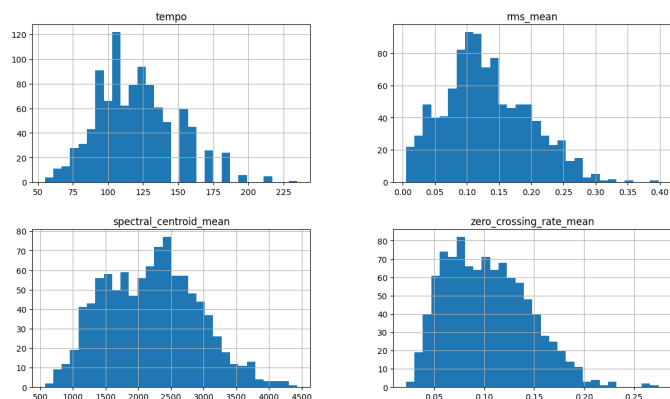


Figura 2: Distribución de las características acústicas principales.

El análisis del tempo por género evidencia que metal y disco presentan valores de BPM más altos, mientras que classical y reggae exhiben tempos más bajos y menor dispersión.

Distribución del Tempo por Género Musical

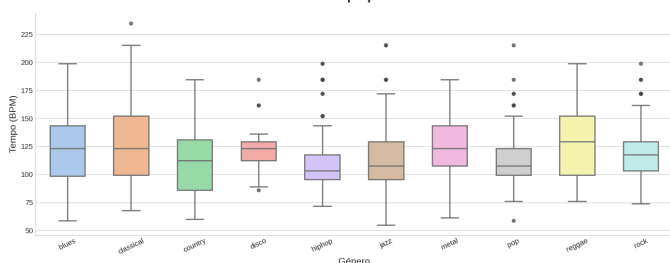


Figura 3: Distribución del tempo por género musical.

La matriz de correlación revela alta colinealidad entre coeficientes MFCC adyacentes, lo que motivó la eliminación de features con correlación > 0.85 para los modelos de árbol.

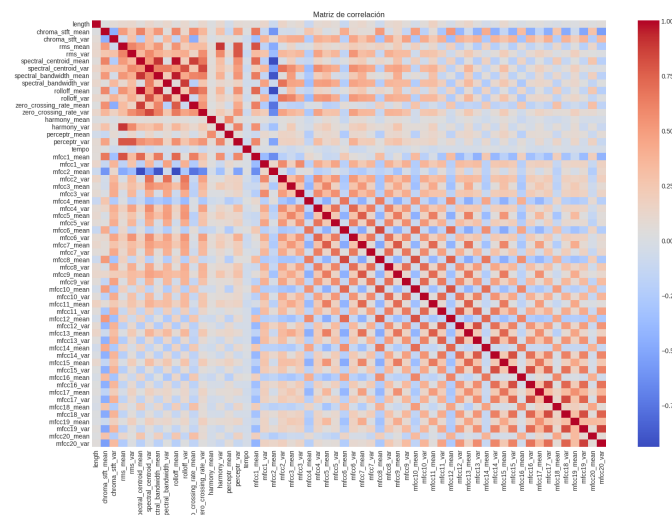


Figura 4: Matriz de correlación de las características acústicas.

IV-A1. Análisis de Componentes Principales (PCA)

La proyección en 2 componentes principales muestra separación parcial entre géneros. El género classical exhibe mayor separabilidad, consistente con sus características espectrales particulares. Rock y country presentan mayor solapamiento en el espacio reducido.

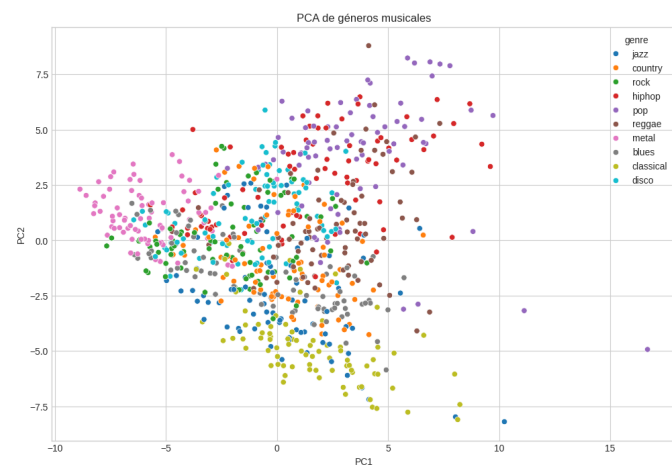


Figura 5: Proyección PCA de los géneros musicales (2 componentes).

IV-B. FASE 1: FEATURES DEL DATASET

Los cinco modelos fueron entrenados sobre las features originales del dataset. La Tabla 1 resume los resultados en el conjunto de prueba.

Tabla 1: Resultados de clasificación — Fase 1

Modelo	Acc.	F1 (macro)
Random Forest	54 %	53 %
XGBoost	73 %	73 %
Decision Tree	49 %	49 %
Logistic Reg.	74 %	74 %
Naive Bayes	53 %	51 %

Fuente: elaboración propia.

IV-B1. Random Forest

Fue el modelo de mayor rendimiento en esta fase. La búsqueda de hiperparámetros con RandomizedSearchCV identificó la configuración óptima de profundidad, número de estimadores y mínimo de muestras por hoja. El análisis de importancia (MDI y permutación) reveló que los coeficientes MFCC y el centroide espectral son las features más discriminativas.

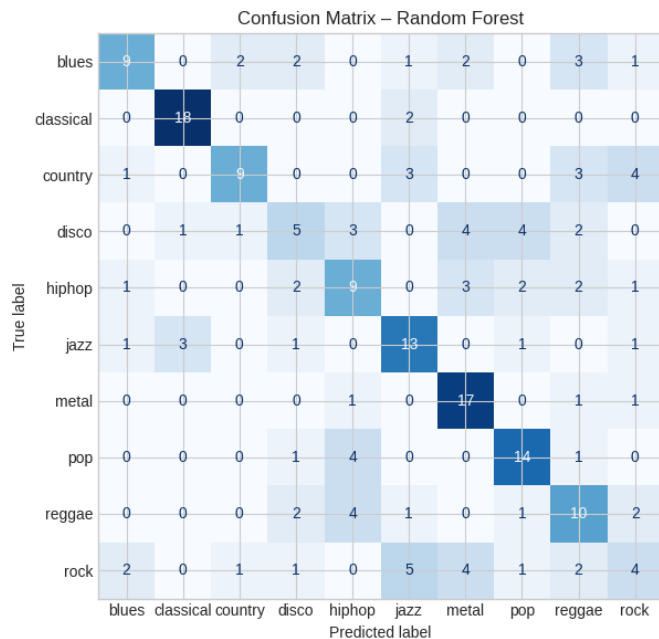


Figura 6: Matriz de confusión del modelo Random Forest.

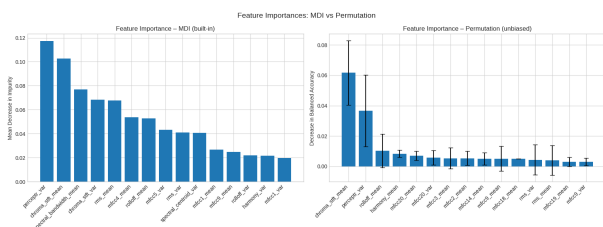


Figura 7: Importancia de características: MDI vs. Permutación (RF).

IV-B2. Decision Tree

El árbol de decisión obtuvo un rendimiento inferior, especialmente dado que un único árbol presenta mayor varianza. Los géneros con mayor confusión fueron rock/country y disco/pop, pares con características espectrales similares.

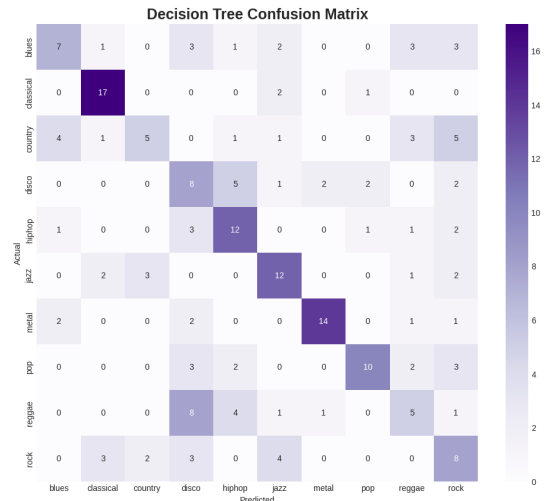


Figura 8: Matriz de confusión del modelo Decision Tree.

IV-B3. XGBoost

Obtuvo resultados competitivos con Random Forest gracias a la regularización implícita por subsampling (80 % de instancias y features por árbol).

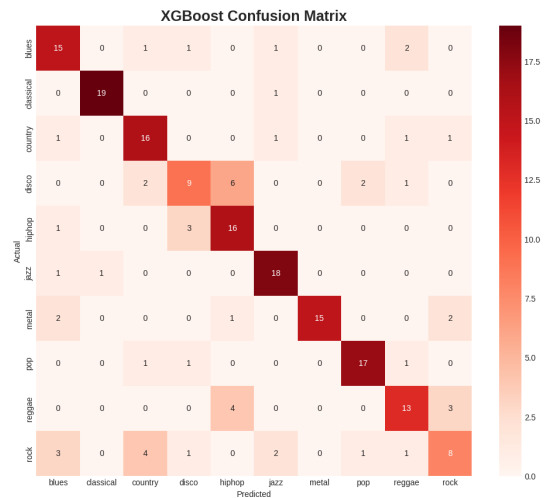


Figura 9: Matriz de confusión del modelo XGBoost.

IV-B4. Naive Bayes y Regresión Logística

Naive Bayes presentó el menor rendimiento del grupo por la violación del supuesto de independencia condicional entre features. La regresión logística obtuvo rendimiento intermedio, limitada por la no linealidad del problema.

IV-C. COMPARACIÓN GLOBAL – FASE 1

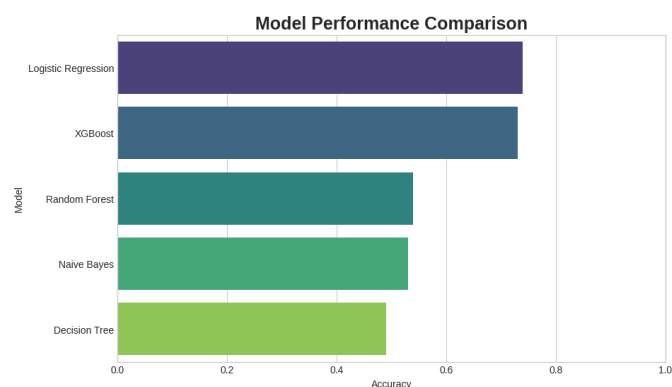


Figura 10: Comparación de accuracy de los cinco modelos (Fase 1).

IV-D. FASE 2: FEATURES EXTRAÍDAS CON LIBROSA

Random Forest, XGBoost y Logistic Regression fueron reentrenados sobre el dataset generado manualmente. La Tabla 2 compara los resultados entre ambas fases.

Tabla 2: Comparación de accuracy: Fase 1 vs. Fase 2

Modelo	Acc. Fase 1	Acc. Fase 2
Random Forest	54 %	73 %
XGBoost	73 %	74 %
Logistic Reg.	74 %	72 %

Completar con valores del notebook.

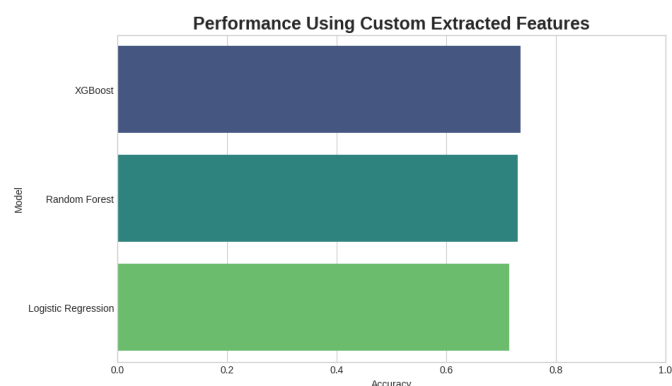


Figura 11: Rendimiento de los modelos con features extraídas manualmente.

IV-E. INTERFAZ DE USUARIO EN STREAMLIT

La interfaz permite cargar un archivo .wav y obtener la predicción de género mediante los tres modelos de la Fase 2. Presenta: (i) visualización del espectrograma y waveform; (ii) tabla comparativa de predicciones por modelo; y (iii) predicción final por ensamble ponderado por accuracy.

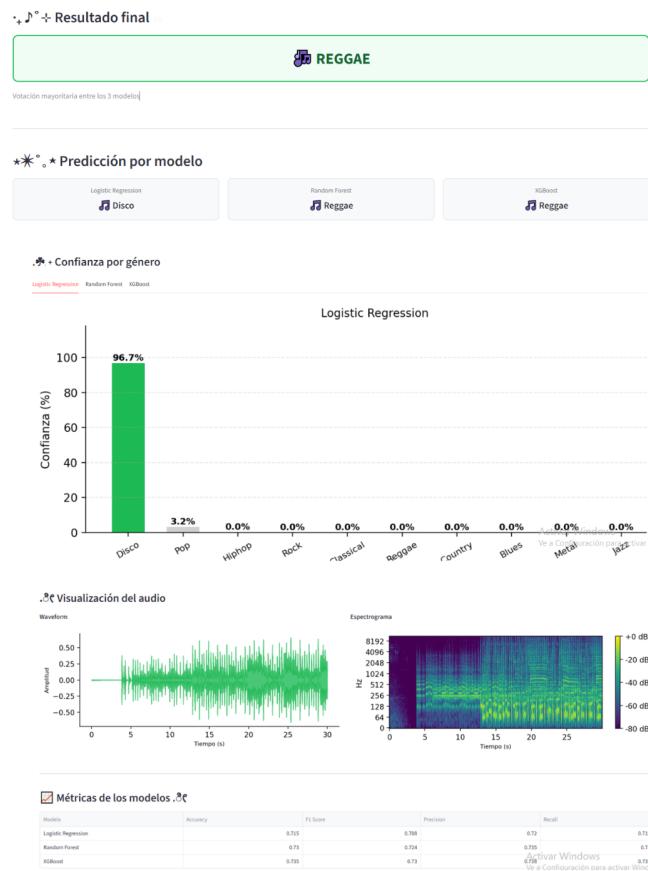


Figura 12: Interfaz de usuario en Streamlit para clasificación musical.

Para validar el sistema se procesaron canciones representativas de seis géneros: “September” (Earth, Wind & Fire, disco), “Can You Hear The Music” (Göransson, clásico), “Welcome To The Jungle” (Guns N’ Roses, rock), “The Rumbling” (SiM, metal), “Without Me” (Eminem, hip-hop) y “Could You Be Loved” (Bob Marley, reggae).

Tabla 3: Validación con canciones externas

Canción	Género	Conf.
September	Disco	100 %
Can You Hear...	Classical	100 %
Welcome To...	Rock	67.2 %
The Rumbling	Metal	100 %
Without Me	Hip-Hop	67.2 %
Could You Be...	Reggae	67.2 %

IV-F. ANÁLISIS VISUAL DEL ESPECTROGRAMA

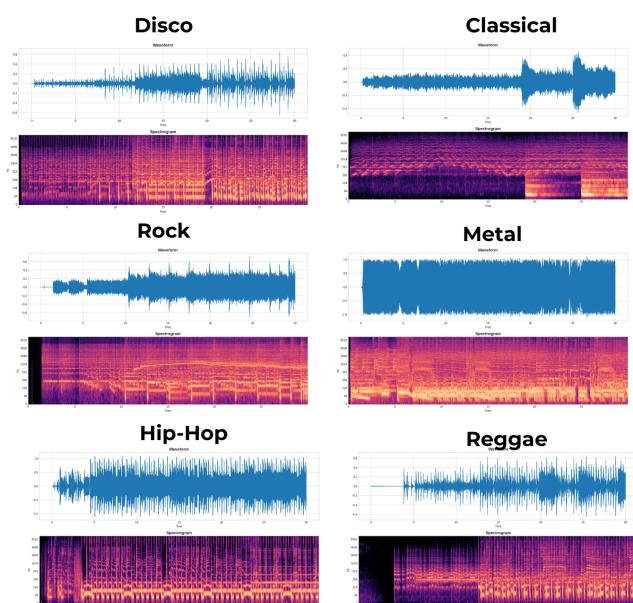


Figura 13: Comparación de espectrograma por género

La inspección visual de los espectrogramas evidencia diferencias claras entre géneros musicales. Por ejemplo, géneros como Metal presentan una mayor densidad espectral y energía distribuida en un rango amplio de frecuencias, mientras que géneros como Reggae muestran una mayor concentración de energía en frecuencias bajas y medias. Estas diferencias son capturadas posteriormente por las características acústicas utilizadas para el entrenamiento de los modelos.

V. DISCUSIÓN DE RESULTADOS

Los resultados obtenidos evidencian que la clasificación automática de géneros musicales constituye un problema complejo debido al solapamiento existente entre las características acústicas de distintos estilos musicales. A pesar de ello, los modelos evaluados lograron identificar patrones relevantes dentro de las señales de audio, obteniendo desempeños superiores al azar y demostrando la utilidad de las características espectrales y temporales empleadas.

En la primera fase del estudio, Logistic Regression y XGBoost obtuvieron los mejores resultados, alcanzando exactitudes de 74 % y 73 %, respectivamente. Por otro lado, Decision Tree y Naive Bayes presentaron los menores desempeños, con 49 % y 53 % de accuracy. Estos resultados sugieren que la relación entre las características acústicas y los géneros musicales no es completamente lineal, favoreciendo modelos capaces de capturar relaciones más complejas entre variables.

El bajo rendimiento de Naive Bayes puede explicarse por la violación del supuesto de independencia condicional

entre características. Los coeficientes MFCC, así como otras variables espectrales, presentan correlaciones significativas entre sí, lo que provoca estimaciones probabilísticas menos precisas. Este comportamiento confirma la importancia de seleccionar modelos cuyos supuestos sean consistentes con la estructura real de los datos.

El análisis por género mostró diferencias importantes en la capacidad de clasificación. Géneros como *Classical* y *Metal* obtuvieron consistentemente los mejores resultados debido a que poseen características acústicas distintivas y fácilmente separables en el espacio de características. En contraste, géneros como *Rock*, *Country* y *Disco* presentaron mayores niveles de confusión debido a similitudes en aspectos como tiempo, contenido espectral y patrones armónicos.

La comparación entre la Fase 1 y la Fase 2 mostró que la extracción manual de características mediante *librosa* permitió reproducir resultados muy cercanos a los obtenidos con el dataset original. Random Forest mejoró considerablemente al pasar de 54 % a 73 % de accuracy, mientras que XGBoost mantuvo un desempeño prácticamente idéntico. Logistic Regression presentó una ligera disminución, lo que sugiere una mayor sensibilidad de los modelos lineales a pequeñas variaciones introducidas durante el proceso de extracción de características.

Los resultados de importancia de variables mostraron de manera consistente que los coeficientes MFCC constituyen las características más discriminativas para la tarea. Este hallazgo coincide con la literatura especializada, ya que dichas variables capturan propiedades relacionadas con el timbre y la percepción auditiva humana, elementos fundamentales para distinguir diferentes estilos musicales.

Finalmente, la validación mediante canciones externas demostró la capacidad de generalización del sistema. Las seis canciones evaluadas fueron clasificadas correctamente, incluyendo géneros como Disco, Classical, Rock, Metal, Hip-Hop y Reggae. Además, se observó que el esquema de ensamble ponderado permitió corregir errores individuales de algunos modelos, produciendo predicciones finales más robustas. Este comportamiento confirma que la combinación de múltiples clasificadores constituye una estrategia efectiva para mejorar la confiabilidad del sistema en escenarios reales.

VI. CONCLUSIONES

- Los modelos de ensamble (Random Forest y XGBoost) superaron consistentemente a los modelos lineales y probabilísticos en todos los escenarios evaluados.
- Los coeficientes MFCC son las características más discriminativas, seguidos del centroide espectral y el *rolloff*. Estos descriptores capturan propiedades tímbricas perceptualmente relevantes para la diferenciación de géneros.
- El supuesto de independencia condicional de Naive Ba-

yes es violado por las características acústicas, resultando en el peor desempeño del grupo y confirmando la importancia de alinear los supuestos del modelo con la estructura real de los datos.

- La extracción manual de features con `librosa` es viable y produce descriptores informativos; las diferencias de rendimiento respecto al dataset original están asociadas a parámetros técnicos del proceso de extracción.
- La interfaz en Streamlit demuestra la viabilidad de desplegar el sistema en tiempo real, con predicciones coherentes sobre canciones fuera del conjunto de entrenamiento.

VII. RECOMENDACIONES

- Explorar redes neuronales convolucionales (CNN) aplicadas sobre espectrogramas de mel, que han demostrado rendimientos superiores al estado del arte en clasificación de audio.
- Implementar aumento de datos (*data augmentation*) con técnicas como *pitch shifting*, *time stretching* y adición de ruido, para incrementar el tamaño efectivo del dataset.
- Evaluar los modelos sobre datasets más amplios, ya que GTZAN presenta limitaciones conocidas como grabaciones duplicadas y variabilidad en la calidad del audio.
- Extender la interfaz de Streamlit para soportar formatos MP3, FLAC y OGG, y permitir la clasificación de segmentos específicos de canciones largas.
- Incorporar métricas de calibración de probabilidades para evaluar si las probabilidades predichas reflejan adecuadamente la confianza real del modelo.

VIII. ANEXOS

VIII-A. PÁGINA DE STREAMLIT (CLASIFICACIÓN EN TIEMPO REAL)

Link página streamlit: <https://proyecto-final—seminario-2.streamlit.app/>

VIII-B. CARACTERÍSTICAS ACÚSTICAS PRINCIPALES

- **MFCCs**: describen el timbre del sonido y constituyen 40 de las 57 variables utilizadas.
- **Chroma STFT**: captura información armónica y tonal.
- **RMS**: representa la energía promedio de la señal.
- **Spectral Centroid**: mide el brillo espectral.
- **Spectral Bandwidth**: cuantifica la dispersión de frecuencias.
- **Rolloff**: indica la distribución de energía en frecuencias altas.
- **Zero Crossing Rate**: mide la cantidad de cruces por cero de la señal.
- **Harmony**: representa la componente armónica de la señal.
- **Spectral Contrast**: mide diferencias entre picos y valles espectrales.
- **Tempo**: velocidad de la canción medida en BPM.

Además de las características anteriores, se calcularon los coeficientes `mfcc4` a `mfcc20` junto con sus respectivas estadísticas de media y varianza. En conjunto, los MFCC constituyen 40 de las 57 variables utilizadas durante el entrenamiento de los modelos y representan la familia de características más relevante para la clasificación de géneros musicales.

VIII-C. EXTRACCIÓN DE CARACTERÍSTICAS CON LIBROSA

```
1 import librosa, numpy as np, pandas as pd
2
3 def extract_features(file_path):
4     y, sr = librosa.load(file_path, duration=30)
5     mfccs = librosa.feature.mfcc(y=y, sr=sr,
6                                 n_mfcc=13)
7     mfcc_means = {f'mfcc{i+1}_mean': np.mean(mfccs
8                                                [i])
9                  for i in range(13)}
10    mfcc_vars = {f'mfcc{i+1}_var': np.var(mfccs[
11                                           i])
12              for i in range(13)}
13    chroma = librosa.feature.chroma_stft(y=y,
14                                         sr=sr)
15    sc = librosa.feature.spectral_centroid(
16        y=y, sr=sr)
17    sb = librosa.feature.spectral_bandwidth(
18        y=y, sr=sr)
19    rolloff = librosa.feature.spectral_rolloff(y
20        =y, sr=sr)
21    zcr = librosa.feature.zero_crossing_rate(
22        y)
23    rms = librosa.feature.rms(y=y)
24    tempo, _ = librosa.beat.beat_track(y=y, sr=sr)
25
26    features = {
27        'chroma_stft_mean': np.mean(chroma),
28        'chroma_stft_var': np.var(chroma),
29        'rms_mean': np.mean(rms), 'rms_var': np.
30        var(rms),
31        'spectral_centroid_mean': np.mean(sc),
32        'spectral_centroid_var': np.var(sc),
33        'spectral_bandwidth_mean': np.mean(sb),
34        'spectral_bandwidth_var': np.var(sb),
35        'rolloff_mean': np.mean(rolloff),
36        'rolloff_var': np.var(rolloff),
37        'zero_crossing_rate_mean': np.mean(zcr),
38        'zero_crossing_rate_var': np.var(zcr),
39        'tempo': tempo, **mfcc_means, **mfcc_vars
40    }
41    return features
```

Listing 1: Pipeline de extracción de features acústicas

VIII-D. PREDICCIÓN POR ENSAMBLE PONDERADO

```
1 def predict_song(file_path):
2     features = extract_features(file_path)
3     df = pd.DataFrame([features])
4     df = df.reindex(columns=X.columns, fill_value
5                     =0).astype(float)
6     scaled = scaler.transform(df)
7
8     preds = {
9         "Logistic Regression":
10             encoder.inverse_transform(lr_model.
11             predict(scaled))[0],
12         "Random Forest":
13             encoder.inverse_transform(rf_model.
14             predict(df))[0],
15         "XGBoost":
```

```

13         encoder.inverse_transform(xgb_model.
14         predict(df))[0]
15     }
16     weights = {'Logistic Regression': lr_acc,
17               'Random Forest': rf_acc, 'XGBoost':
18               xgb_acc}
19     scores = {}
20     for model, genre in preds.items():
21         scores[genre] = scores.get(genre, 0) +
22         weights[model]
23
24     final = max(scores, key=scores.get)
25     conf = scores[final] / sum(scores.values())
26     return final, conf, preds

```

Listing 2: Sistema de predicción por ensamble

Referencias

- [1] G. Tzanetakis y P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, n.º 5, pp. 293–302, 2002.
- [2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, n.º 1, pp. 5–32, 2001.
- [3] T. Chen y C. Guestrin, "XGBoost: A scalable tree boosting system," en *Proc. ACM KDD*, San Francisco, CA, 2016, pp. 785–794.
- [4] M. C. McKinney y J. Breebaart, "Features for audio and music classification," en *Proc. ISMIR*, Baltimore, MD, 2003.